Programski prevodioci

11 jvm

25-26

Dunja Vrbaški

Fakultet tehničkih nauka, Novi Sad

Izvršno okruženje

Podsetnik:

Generisanje koda, faza sinteze
→ neophodno okruženje u okviru kog će se program izvršavati
Prevođenje je uvek u skladu sa okruženjem.

Model izračunavanja

Okruženje je podrška za
- organizaciju i upravljanje memorijom
- upravljanje pozivima funkcija
- upravljanje apstrakcijama jezika

Programski jezik je definisan sintaksom i semantikom u odnosu na apstraktnu mašinu.

Prevodilac prevodi u odnosu na izvršno okruženje

- konkretna mašina (CPU registri, RAM, instrukcije, OS interfejes)
- virtuelna mašina (softver)

Virtuelna mašina

Virtuelna mašina je zapravo program (softver) koji se ponaša kao mašina (računar).
Omogućava izvršavanje apstrakcija.

Postoje stanja (upravlja se različitim podacima koji modeliraju memoriju) i postoje instrukcije (svoj poseban skup instrukcija)
Sve je to "izgrađeno od softvera", nema veze sa organizacijom memorije i instrukcijama na konkretnoj mašini.

Primeri:
JVM – Java Virtual Machine
CLR – Common Language Runtime (.net)

Prednosti:
- nezavisnost
- sigurnost
- portabilnost
- optimizacije

Virtuelna mašina

- *sistem (računar i operativni sistem) VS*
- *proces (izvršavanje programa)*

Dva osnovna tipa VM u odnosu na izvršavanje operacija:

Stack-based VM

- lokacija operanada i rezultata nije eksplicitno naznačena
- uvek je na vrhu steka
- sve se realizuje preko operacija sa stekom
- tri instrukcije su potrebne da se izvrši sabiranje
- jednostavnije instrukcije, manje parametara
- brže i jednostavnije za implementaciju
- *Java VM, .NET CLR*

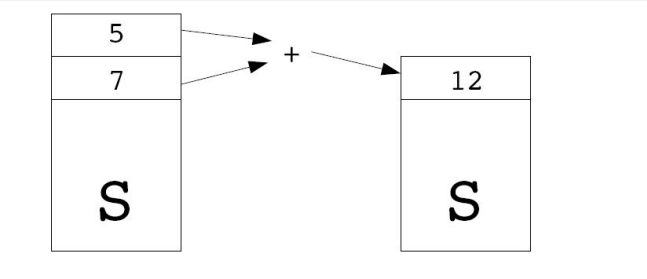
Register-based VM

- eksplicitno navođenje adresa (operandi su u registrima)
- jedna instrukcija za sabiranje
- bliže fizičkim mašinama
- kod je manje kompaktan
- često potrebne određene pretpostavke o CPU
- određene optimizacije moguće
- *Android Runtime Environment (ART)*

Računar sa stek arhitekturom

- jedino skladište je stek
- forma instrukcije $r = F(a_1, \dots, a_n)$
    - F je funkcija, a_i su operandi, r je rezultat
    - POP n operanada sa vrha steka
    - izračunavanje funkcije F korišćenjem tih operanada
    - PUSH rezultat r na vrh steka

```
PUSH 7
PUSH 5
ADD
```



Stek za izračunavanje (evaluation / operand stack)

Evaluacija izraza i prenos vrednosti unutar jedne metode

!=

Stek aktivacionih blokova (call stack)

Modeluje dinamiku poziva metoda

Java Virtual Machine (JVM)

program.java

↓ javac (java compiler)

program.class

↓ JVM

execution

- class fajl je nezavisan od platforme (arhitektura + OS)
- sadrži bytecode – međukod Java Virtuelne Mašine (JVM)
- jezik višeg nivoa od asemblerskog
- bogat metapodacima; dodatne informacija (debug, informacije o tipovima,...)

```java
public class Main {
    public static void main(String[] args) {

        int a, b;
           a = 5;
           b = 3;
           a = a + b;

        System.out.println(a);
    }
}
```

```
public class Main {
  public Main();
    Code:
       0: aload_0
       1: invokespecial #1
       4: return

  public static void main(java.lang.String[]);
    Code:
       0: iconst_5
       1: istore_1
       2: iconst_3
       3: istore_2
       4: iload_1
       5: iload_2
       6: iadd
       7: istore_1
       8: getstatic     #7
      11: iload_1
      12: invokevirtual #13
      15: return
}
```

```
(jdk) javap -c Main.class
```

disassembled code
(neformalni hipotetski asembler)
u primeru iznad nisu navedeni komentari (koji postoje)

JVM Memorijski model

Različiti namenski delovi kao podrška izvršavanju.
U nastavku – okvir, osnovni delovi

Stek
- svaka nit (thread) ima svoj stek na kom će se nalaziti frejmovi (lokalne promenljive, međurezultati, podrška za aktivaciju i povratak iz metoda)

Heap
- sve niti dele ovu oblast
- smeštaju se svi objekti

Programski brojač
- svaka nit ima svoj registar za programski brojač

Method Area

- sve niti dele ovu oblast
- inicijalizuje se na početku
- za svaku klasu/interfejs se čuva bajtkod metoda i runtime constant pool

Constant pool

- string i numeričke konstante
- simboličke reference (članovi drugih klasa/interfejsa; dynamic linking)

Nema registara opšte namene

Naravno, konkretna implementacija JVM interno koristi registre konkretne mašine

```java
String str1 = new String("Dobar dan");
String str2 = new String("Dobar dan");

if (str1 == str2)
      System.out.println("isti");
else
      System.out.println("razliciti");
```

```java
String str1 = "Dobar dan";
String str2 = "Dobar dan";

if (str1 == str2)
      System.out.println("isti");
else
      System.out.println("razliciti");
```

Podrška za aktivaciju i izvršavanje metoda

Stek frejm

- svaki poziv metode kreira frejm na steku
- sadrži (makar):
    - lokalne promenljive
    - operand stack
        - na početku prazan
        - koristi se za računanje (push, pop,...)
        - koristi se i za pripremu argumenata i za preuzimanje povratne vrednosti

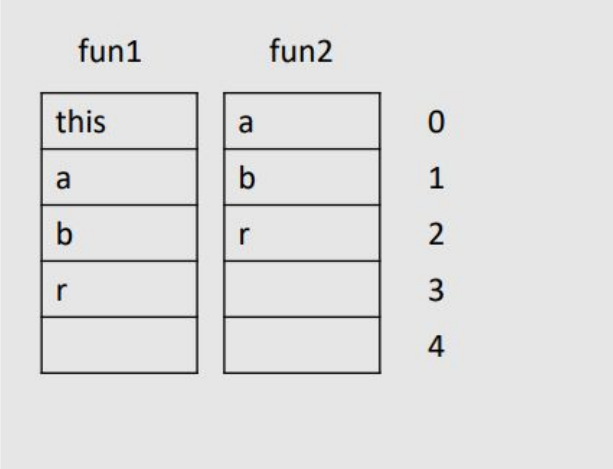
    - pokazivač na CP odgovarajuće klase
        - pristup konstantama i elementima drugih klasa ili interfejsa

Lokalne promenljive

- adresiraju se preko indeksa (prva je 0)
- kod metoda instance prva promenljiva (indeks 0) je “this”
- slede argumenti pa lokalne promenljive
- mogu zauzimati više mesta. long i double, na primer, zauzimaju dva mesta

```
public int fun1(int a, int b) {
   int r = a + b;
   return r;
}

public static int fun2(int a, int b) {
   int r = a + b;
   return r;
}
```



Instrukcije

- opcode + 0 ili više operanada
- instrukcije zavise od tipa – prefiks (iload, fadd,...)

- Čitanje i pisanje (promenljive <-> operand stek)
    - promenljiva na/sa steka: iload, fload, istore, fstore,...
    - konstanta na/sa steka: iconst_0, bipush, ldc,..
    - ...

- Aritmetičke operacije
    - iadd, fadd, isub, fsub, imul, ior, iinc,...

- Rad sa objektima
    - kreiranje: new
    - polja: getstatic, putstatic, getfield, putfield
    - nizovi: newarray, arraylength, iaload, faload, iastore, fastore,...

- Kontrola toka
    - uslovni skok: if_icmpeq, if_icmplt, ifle, ifnull, ...
    - bezuslovni skok: goto...

- Metode
    - invokevirtual, invokeinterface, invokestatic, invokespecial, invokedynamic
    - ireturn, freturn,..., return

- Izuzeci, sinhronizacija, konverzija tipova

```java
public class Main {
    public static void main(String[] args) {

         int a, b;
           a = 5;
           b = 3;
           a = a + b;

         System.out.println(a);
    }
}
```

```
public class Main {
  public Main();
    Code:
       0: aload_0
       1: invokespecial #1                  // Method java/lang/Object."<init>":()V
       4: return

  public static void main(java.lang.String[]);
    Code:
       0: iconst_5
       1: istore_1
       2: iconst_3
       3: istore_2
       4: iload_1
       5: iload_2
       6: iadd
       7: istore_1
       8: getstatic     #7                  // Field java/lang/System.out:Ljava/io/PrintStream;
      11: iload_1
      12: invokevirtual #13                 // Method java/io/PrintStream.println:(I)V
      15: return
}
```

```java
public class Main {
    public static void main(String[] args) {

        int a, b;
          a = 5;
          b = 3;
          a = a + b;

        System.out.println(a);
    }
}
```

```
public class Main {
  public Main();
    Code:
       0: aload_0
       1: invokespecial #1                  // Method java/lang/Object."<init>":()V
       4: return

  public static void main(java.lang.Str
    Code:
       0: iconst_5
       1: istore_1
       2: iconst_3
       3: istore_2
       4: iload_1
       5: iload_2
       6: iadd
       7: istore_1
       8: getstatic     #7                  // Field java/lang/System.out:Ljava/io/PrintStream;
      11: iload_1
      12: invokevirtual #13                 // Method java/io/PrintStream.println:(I)V
      15: return
}
```

konstruktor

referenca (na this, lok prom 0)

void

lok prom 0

push const 5

store sa steka u lok prom 1 (a)

push lok prom 1 (a)

operacija

push lok prom 1 (a) - priprema za invoke

```java
public class Main {
    public static void main(String[] args) {

        int a, b;
           a = 5;
           b = 3;
           a = a + b;

        System.out.println(a);
    }
}
```

```
public class Main
  ...
  this_class: #19                         // Main
  super_class: #2                         // java/lang/Object
  interfaces: 0, fields: 0, methods: 2, attributes: 1
Constant pool:
   #1 = Methodref          #2.#3          // java/lang/Object."<init>":()V
   #2 = Class              #4             // java/lang/Object
   #3 = NameAndType        #5:#6          // "<init>":()V
   ...
   #7 = Fieldref           #8.#9          // java/lang/System.out:Ljava/io/PrintStream;
   ...
   #13 = Methodref          #14.#15        // java/io/PrintStream.println:(I)V
   #19 = Class              #20            // Main
   #20 = Utf8               Main
   ...
{
  public Main();
   ...
   Code:
      stack=1, locals=1, args_size=1
         0: aload_0
         1: invokespecial #1                  // Method java/lang/Object."<init>":()V
         4: return
      LineNumberTable:
        line 1: 0

  public static void main(java.lang.String[]);
    ...
    Code:
      stack=2, locals=3, args_size=1
         0: iconst_5
         1: istore_1
         2: iconst_3
         3: istore_2
         4: iload_1
         5: iload_2
         6: iadd
         7: istore_1
         8: getstatic     #7                  // Field java/lang/System.out:Ljava/io/PrintStream;
        11: iload_1
        12: invokevirtual #13                 // Method java/io/PrintStream.println:(I)V
        15: return
      LineNumberTable:
        line 4: 0
        line 5: 2
        line 6: 4
        line 8: 8
        line 9: 15
}
```

```
javap -c -v Main.class
```

```java
public class MyClass {

   private int x;
   public int y;

   public static int s;

   public int fun1(int a, int b) {
      int r = a + b + x;
      return  r;
   }

   public static int fun2(int a, int b) {
      int r = a + b + s;
      return r;
   }
}
```

```
public class MyClass {
  private int x;

  public int y;

  public static int s;

  public MyClass();
    Code:
       0: aload_0
       1: invokespecial #1                  // Method java/lang/Object."<init>":()V
       4: return

  public int fun1(int, int);
    Code:
       0: iload_1
       1: iload_2
       2: iadd
       3: aload_0
       4: getfield      #7                  // Field x:I
       7: iadd
       8: istore_3
       9: iload_3
      10: ireturn

  public static int fun2(int, int);
    Code:
       0: iload_0
       1: iload_1
       2: iadd
       3: getstatic     #13                 // Field s:I
       6: iadd
       7: istore_2
       8: iload_2
       9: ireturn
}
```

```
javap -c -p Main.class
```

```java
public class MyClass {

   public int fun1(int a) {
      for (int i = 0; i <= 10; i++) {
         a++;
      }
      return  a;
   }
}
```

```
public class MyClass {
  public MyClass();
    Code:
       0: aload_0
       1: invokespecial #1                  // Method java/lang/Object."<init>":()V
       4: return

  public int fun1(int);
    Code:
       0: iconst_0
       1: istore_2
       2: iload_2
       3: bipush        10
       5: if_icmpgt     17
       8: iinc          1, 1
      11: iinc          2, 1
      14: goto          2
      17: iload_1
      18: ireturn
}
```

uzmi 2 sa steka, ako je prvi veci od drugog idi na 17
lok.prom 1 (a) uvecaj za 1
lok.prom 2 (i) uvecaj za 1

iconst: postoje samo od -1 (iconst_m1) do 5 (iconst 5)

```java
public class MyClass {

   public int fun1(int a) {
      for (int i = 0; i <= 10; i++) {
         a++;
      }
      return  a;
   }
}
```

```
public class MyClass {
  public MyClass();
    Code:
       0: aload_0
       1: invokespecial #1                  // Method java/lang/Object."<init>":()V
       4: return

  public int fun1(int);
    Code:
       0: iconst_0
       1: istore_2
       2: iload_2
       3: bipush        10
       5: if_icmpgt     17
       8: iinc          1, 1
      11: iinc          2, 1
      14: goto          2
      17: iload_1
      18: ireturn
}
```

Broj sa leve strane, indeks – bajt ofset od početka metode

Čitljiv format- koriste se mnemonici za instrukcije.
Opcodes su kodirane jednim bajtom.
Neke operacije imaju više operanda i operandi mogu zauzimati više bajtova

iconst_0 -> 1 bajt, operand je enkodiran u samu operaciju (iconst_0: opcode 0x03)

bipush 10 -> 2 bajta, opcode i operand (bipush je za postavljanje konstanti izmedju -127 i 127) (bipush: opcode 0x10)

```java
public class MethodTest {
    public static void main(String[] args) {
        int a = 5;
        System.out.println(sum(a));
    }

    public static int sum(int m) {
        int sum = 0;
        for (int i = 0; i <= m; i++)
        {
            sum += i;
        }
        return sum;
    }
}
```

```
public static void main(java.lang.String[]);
  Code:
     0: iconst_5
     1: istore_1
     2: getstatic     #7
     5: iload_1
     6: invokestatic  #13   // Method sum:(I)I
     9: invokevirtual #19
    12: return

public static int sum(int);
  Code:
     0: iconst_0
     1: istore_1
     2: iconst_0
     3: istore_2
     4: iload_2
     5: iload_0
     6: if_icmpgt     19
     9: iload_1
    10: iload_2
    11: iadd
    12: istore_1
    13: iinc          2, 1
    16: goto          4
    19: iload_1
    20: ireturn
```

```java
public class HVTest {
    public static void main(String[] args) {
       int a = 3, b = 5;
       String str = "Hello World: ";

       System.out.println(str + (a + b));
    }
}
```

```java
public class HVTest {
    public static void main(String[] args) {
       int a = 3, b = 5;
       String str = "Hello World: ";

       System.out.println(str + (a + b));
    }
}
```

```
public class HVTest {
  public HVTest();
    Code:
       0: aload_0
       1: invokespecial #1
       4: return

  public static void main(java.lang.String[]);
    Code:
       0: iconst_3
       1: istore_1
       2: iconst_5
       3: istore_2
       4: ldc           #7
       6: astore_3
       7: getstatic     #9
      10: aload_3
      11: iload_1
      12: iload_2
      13: iadd
      14: invokedynamic #15,  0
      19: invokevirtual #19
      22: return
}
```

vrednost 3 na stek
int vrednost int u promenljivu i

konstanta #7 na stek
referenca u promenljivu 3

saberi i stavi rezultat na stek

Prethodni primeri – čitljiv kod (alat javap).

class fajl – niz bajtova koji ima određenu strukturu.

Sadrži informacije:

- magic number (0xCAFEBABE; Gosling)
- major i minor verzija
- constant pool (broj elemenata i sama tabela)
- access flags (flegovi za: public, abstract, enum,...)
- this class, super class (indexi u CP tabeli)
- interfejsi (broj interfejsa, niz indeksa u CP tabeli)
- polja (broj polja i tabela sa informacijama o poljima)
- metode (broj metoda i tabela sa informacijama o metodama)
- atributi (broj atributa i tabela sa informacijama o atributima)

```
public class HVTest
  minor version: 0
  major version: 59
  ...
Constant pool:
   #1 = Methodref          #2.#3          // java/lang/Object."<init>":()V
   #2 = Class              #4             // java/lang/Object
   #3 = NameAndType        #5:#6          // "<init>":()V
   #4 = Utf8               java/lang/Object
   ...
   #46 = Utf8               java/lang/invoke/MethodHandles
   #47 = Utf8               Lookup
{
  public HVTest();
    descriptor: ()V
    flags: (0x0001) ACC_PUBLIC
    Code:
      stack=1, locals=1, args_size=1
         0: aload_0
         1: invokespecial #1                  // Method java/lang/Object."<init>":()V
         4: return
      LineNumberTable:
        line 1: 0
        ...
```

parametri i povratna vrednost

```
  public static void main(java.lang.String[]);
    descriptor: ([Ljava/lang/String;)V
    flags: (0x0009) ACC_PUBLIC, ACC_STATIC
    Code:
      stack=4, locals=4, args_size=1
         0: iconst_3
         1: istore_1
         2: iconst_5
         3: istore_2
         4: ldc           #7        // String Hello World:
         6: astore_3
         7: getstatic     #9        // Field java/lang/System.out:Ljava/io/PrintStream;
        10: aload_3
        11: iload_1
        12: iload_2
        13: iadd
        14: invokedynamic #15,  0  // InvokeDyn[illegible]makeConcatWithConstants:(Ljava/lang/String;I)Ljava/lang/String;
        19: invokevirtual #19      // [illegible]java/io/PrintStream.println:(Ljava/lang/String;)V
        22: return
      LineNumberTable:
        line 4: 0
        line 5: 4
        line 7: 7
        line 8: 22
}
...
```

veza sa izvornim kodom: liniji 4 u SC odogovara linija 0 ovde

```
1       public class HVTest {
2           public static void main(String[] args) {
3
4                       int a = 3, b = 5;
5                       String str = "Hello World: ";
6
7                       System.out.println(str + (a + b));
8           }
9       }
```

```
public class HVTest {
  public HVTest();
    Code:
       0: aload_0
       1: invokespecial #1
       4: return

  public static void main(java.lang.String[]);
    Code:
       0: iconst_3
       1: istore_1
       2: iconst_5
       3: istore_2
       4: ldc           #7
       6: astore_3
       7: getstatic     #9
      10: aload_3
      11: iload_1
      12: iload_2
      13: iadd
      14: invokedynamic #15,  0
      19: invokevirtual #19
      22: return
}
```

javap –c Main.class

```
...

Constant pool:
   #1 = Methodref          #2.#3
   #2 = Class              #4
   #3 = NameAndType        #5:#6
   #4 = Utf8               java/lang/Object
   #5 = Utf8               <init>
   #6 = Utf8               ()V
   #7 = String             #8
   #8 = Utf8               Hello World:
   #9 = Fieldref           #10.#11
   #10 = Class             #12
   #11 = NameAndType       #13:#14

   ...

   #46 = Utf8              java/lang/invoke/MethodHandles
   #47 = Utf8              Lookup
{
...
```

javap –c -v Main.class

```
...

Constant pool:
   #1 = Methodref          #2.#3
   #2 = Class              #4
   #3 = NameAndType        #5:#6
   #4 = Utf8               java/lang/Object
   #5 = Utf8               <init>
   #6 = Utf8               ()V
   #7 = String             #8
   #8 = Utf8               Hello World:
   #9 = Fieldref           #10.#11
   #10 = Class             #12
   #11 = NameAndType       #13:#14

   ...
  public static void main(java.lang.String[]);
  ...
  Code:
       ...
       4: ldc           #7
       6: astore_3
       7: getstatic     #9
      10: aload_3
      11: iload_1
```

Constant pool

- uređen skup konstanti koje određeni tip koristi
- sve potrebno za izvršavanje svega definisanog u nekoj klasi ili interfejsu
- simboličke reference preko kojih se povezujemo sa drugim entitetima

```
- getstatic     #9   -> System.out.PrintStream
- ldc           #7   -> Hello World:
```

- opis KAKO da se nešto pronađe
  još smo u fajlu, nismo u memoriji

```
...

Constant pool:
   #1 = Methodref          #2.#3
   #2 = Class              #4
   #3 = NameAndType        #5:#6
   #4 = Utf8               java/lang/Object
   #5 = Utf8               <init>
   #6 = Utf8               ()V
   #7 = String             #8
   #8 = Utf8               Hello World:
   #9 = Fieldref           #10.#11
   #10 = Class             #12
   #11 = NameAndType       #13:#14

   ...
  public static void main(java.lang.String[]);
  ...
  Code:
       ...
       4: ldc           #7
       6: astore_3
       7: getstatic     #9
      10: aload_3
      11: iload_1
```

simbolička referenca = tip

```
Methodref        Class.NameAndType
Class            ime klase
NameAndType      ime:tip  (V - void)
Utf8             niz bajtova
String           referenca ka sadržaju stringa
Fieldref         Class.NameAndType

Integer, Float, Double, Long
Methodref, InterfaceMethodref
```

```
...

Constant pool:
   #1 = Methodref          #2.#3          // java/lang/Object."<init>":()V
   #2 = Class              #4             // java/lang/Object
   #3 = NameAndType        #5:#6          // "<init>":()V
   ...
   #7 = String             #8             // Hello World:
   #8 = Utf8               Hello World:
   #9 = Fieldref           #10.#11        // java/lang/System.out:Ljava/io/PrintStream;
  #10 = Class              #12            // java/lang/System
  #11 = NameAndType        #13:#14        // out:Ljava/io/PrintStream;
  ...
  #15 = InvokeDynamic      #0:#16         //  #0:makeConcatWithConstants:(Ljava/lang/String;I)Ljava/lang/String;
  #16 = NameAndType        #17:#18        // makeConcatWithConstants:(Ljava/lang/String;I)Ljava/lang/String;
  ...
  #19 = Methodref          #20.#21        // java/io/PrintStream.println:(Ljava/lang/String;)V
  #20 = Class              #22            // java/io/PrintStream
  #21 = NameAndType        #23:#24        // println:(Ljava/lang/String;)V
  ...
  #25 = Class              #26            // HVTest
  #26 = Utf8               HVTest
  #27 = Utf8               Code
  #28 = Utf8               LineNumberTable
  ...
  #31 = Utf8               SourceFile
  #32 = Utf8               HVTest.java
  …
```

- Urađeno je kompajliranje u bytecode

- Sledi izvršavanje
- Izvršavanje počinje od kreiranja inicijalne klase i poziva se main metod
- dinamički se radi učitavanje i povezivanje klasa
- između ostalog formira se runtime CP na osnovu CP iz class fajla
- Otkrivanje i povezivanje imena
  resolution: simbolička → konkretna referenca
  najčešće kod prvog korišćenja
  opis → JVM traži → adresa runtime strukture

Ko ovo radi?

Potrebna je implementacija:

- Java bytecode interpreter i/ili
- JIT kompajler za konkretnu mašinu

JIT kompajler

JIT – *Just in time* kompajler – prevođenje tokom izvršavanja

Iskoristiti prednosti dva tradicionalna pristupa:

AOT – *Ahead of time* kompajler

Interpreter

Proces:

- Posmatraju se delovi koda (klase, metode, petlje,...); dinamčka analiza programa
- Vodi se evidencija o izvršavanju, značajnim delovima u programu (hot spots)
- JVM odlučuje koji delovi su pogodni za kompilaciju na ciljni (mašinski) kod
- Vodi se evidencija o prevedenim delovima
- Kad je potreban određeni deo – umesto interpretera koristi se prevedeni kod

- JIT se obično izvršava kao poseban proces pa ne utiče na proces izvršavanja

- Kad je neki deo bajtkoda preveden - taj deo postaje dostupan
- OSR (On Stack Replacement)
  - prelaz sa interpretiranja na kompajliranu verziju.
  - Ili sa jedne verzije koda na drugu.
  - Može biti prelaska i u toku petlje, nakon određenog broja iteracija, kad je kompajlirana verzija gotova.

- Nije učestalost poziva jedini kriterijum

- Prednost JIT prevođenja je i u mogućnostima dinamičkih optimizacija
  - primer: Retko se poziva metod, ali ima deo koda koji se dobro može optimizovati → kompajlira se
  - primer: Može dodatno da se optimizuje ili da se promeni optimizacija (zbog nekih pogrešnih pretpostavki ili nepotrebnih optimizacija)

- bytecode je ulaz u optimizacionu mašinu

Prednosti:

- ubrzavanje izvršavanja
- korišćenje detalja specifične arhitekture
- dostupnost inforacija koje su poznate samo u runtime-u
- poboljšane optimizacije
- što duže radi brže će se izvršavati?

Mane:

- dodatno vreme za kompilaciju
- povećano korišćenje memorije
- zavisi od toga koliko i kada se delovi kompajliraju
- merenje performansi - obratiti pažnju

- Dinamički se vrše procesi: merenja (profiling), odlučivanja, prevođenja
- Zbog stalnog merenja i pristupa runtime informacijama – mogu bolje optimizacije da se izvrše nego kod tradicionalnog kompajliranja
- Neke VM ni nemaju interpreter, sve metode se JIT kompajliraju
- Kod drugih – može se uključiti/isključiti ili postoje opcije ili mehanizmi za kontrolu JIT procesa

Terminologija:

Za interpreter često kažemo da je "implementacija jezika"
Za kompajler ređe kažemo da je "implementacija jezika"

Za sve jezike, kad napominjemo, često kažemo: "... zavisi od implementacije"

Na primer:

CPython je implementacija jezika Python realizovana u jeziku C.
GCC je kompajler za C.
Koliko će zauzimati long vrednosti zavisi od implementacije.
JIT kompajler.

Kad se definiše VM (npr JVM) – ne znači da nju mora i može koristiti samo određeni jezik, npr Java

Načelno, kao što Java program može da se prevede na bilo šta drugo (umesto java bytecode) i može da koristi drugačije okruženje tako i bilo koji drugi jezik može da koristi JVM.

JVM (bytecode):

- Java
- Scala
- Kotlin
- Clojure
- Jython
- …

CLR (CIL):

- C#
- C++/CLI
- F#
- Eiffel
- IronPyhton
- …

CLR – Common Language Runtime; CIL – Common Intermediate Langage; .net

Pojmovi: interpreted / compiled languages

Iako koristimo termine za jezike – zapravo ne postoji ovakva podela

- mogu se napraviti različite implementacije
- prilikom prevođenja mogu se koristiti (i koriste se) oba pristupa
- određeni delovi se svakako izvršavaju u nekoj meri (front-end, analiza)
- eksplicitno ili implicitno prevođenje u međukod
- interaktivno izvršavanje

Sve ovo može da zbuni i neprecizno definiše jezik kao ovakav ili onakav.

Upravljanje memorijom

Dinamičko zauzimanje memorije

malloc, new,... // free, delete...

Zadaci:

- voditi računa koliko je zauzeto/slobodno
- zauzimati tražene delove memorije
- tražiti od OS inicjalno i proširenje po potrebi
- oslobađati određene delove memorije
- ...

Ciljevi:

- Ispravno upravljati memorijom
- Optimizovati zauzetu količinu i pristup
- Optmizacija samih procesa zauzimanja i oslobađanja
- optimalna upotreba - brže izvršavanje

Dva osnovna zadatka:

- Zauzimanje / alociranje

- Ponovna upotreba / dealociranje (free, delete, GC)

  Dva osnovna pristupa:

  - Ručno (malloc/free)
  - Automatski (GC)

Preporuke:

- osloboditi sve što više nije potrebno (1)
- ne pristupati onome što je oslobođeno + ne oslobađaj prerano (2)
- ne kreirati ono što ti ne treba (3)
- ...

(1) → memory leaks – ne mora biti problem, a može biti veliki problem
(2) → problem je svakako
ne postoji – greška; nešto drugo na tom mestu – (gora) greška
(3) → previše zauzimanja (na primer, bespotrebni copy ctors)

Svaki PL i njegova implementacija - posebni problemi

- Intenzivnim ponavljanjem zauzimanja i oslobađanja - može se doći do značajne fragmentacije

  Rezultat: nema memorije → zapravo je ima, ali nije dobro iskorišćena

- Zauzeta memorija je široko rasuta

- Nema velikih blokova slobodne memorije

- Obično oba procesa, na neki način, vode računa o tome.
  Ili se makar razmatra efikasnost procesa u odnosu na fragmentaciju.

Alokacija

- počinje se od velike količine slobodne memorije (uzastopne lokacije)
- memorija se zauzima vrednostima različite veličine
- neki delovi se oslobađaju
- → nastaju prazna mesta (rupe)
- prilikom novog zauzimanja traži se odgovarajuće mesto

- prilikom novog zauzimanja traži se odgovarajuće mesto
- kako da znamo gde ima slobodnog mesta?

  Na primer, lista pokazivača na slobodne prostore

  Usput: gde bi se čuvala takva lista?

- od *n* mogućih slobodnih prostora – koji izabrati?

  - first-fit: prvi koji je dovoljno velik
  - best-fit: najmanji mogući deo koji je dovoljno velik
  - zauzimaju se samo određeni blokovi (npr veličine su stepeni broja 2)

- Kakav će biti novi raspored slobodnog prostora?
- Stari i novi objekti pomešani?
- Ko ovo radi? Da li neki memory manager ili programer?

- Da li je moguće preraspodeliti? Da li to raditi svaki put ili ponekad?

Garbage collection

“Automatsko” oslobađanje (recikliranje) memorije u izvršnom okruženju.

Razlikuje objekte koji su dostupni od onih koji više neće biti korišćeni i bezbedno oslobađa memoriju.

Problemi:

- (ranije) lošije implementacije: sporo ukupno izvršavanje, velike pauze,
- fragmentacije
- situacije teže za detektovanje: dostupan objekat, ali se neće koristiti

Dva procesa, ne nužno funkcionalno odvojena:

- mehanizam za detekciju živih objekata
- mehanizam za samo “oslobađanje” memorije

- Najčešće se detektuju, na neki način, živi objekti u smislu dostupnosti.
- Moguće i slabiji algoritam u odnosu na analizu koju može da vrši kompajler. Aproksimacija zadatka.
- Oslobađanje - izmena evidencije o tome gde je slobodna memorija

Idtorijski, dva glavna pristupa/ideje:

- reference counting
- trace-based GC

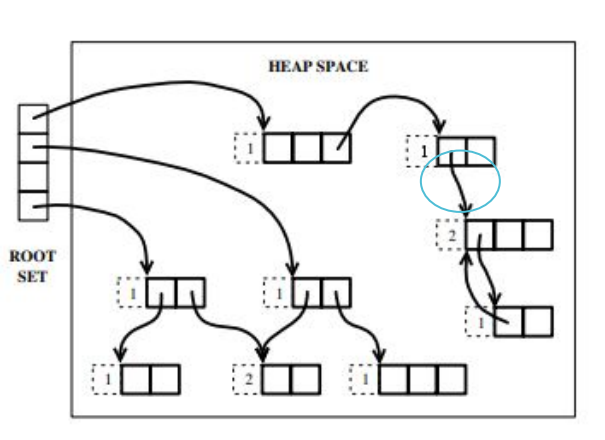
reference counting

Objekat je živ ako neko na njega pokazuje.

- broje se reference
- kad se pojavi neka nova referenca pojavi - poveća se brojač
- kada preston neka refrenca da pokazuje (izađe iz opsega ili se prebci na drugi objekat) - smanji se brojač
- Kada je broj referenci 0 – objekat je nedostupan – kandidat za brisanje

- uvek se zna status jednog objekta
- brzo
- zauzima dodatne resurse (mesto za brojanje referenci)
- nike kompatibilan sa nekim JIT optimizacijama

- problem sa cikličnim vezama
  (primer gde je razlika između realno živog objekta i onog koji detektuje gc)



Wilson, Paul R. "Uniprocessor garbage collection techniques."

```java
class Osoba {
    String ime;
    Osoba partner; // Referenca na drugu instancu klase Osoba

    Osoba(String ime) {
        this.ime = ime;
    }

}

public class Main {
    public static void napraviCiklus() {
        Osoba ana = new Osoba("Ana");
        Osoba bojan = new Osoba("Bojan");

        ana.partner = bojan;
        bojan.partner = ana;
    }

    public static void main(String[] args) {
        napraviCiklus();
    }
}
```

```java
A a = new A();
B b = new B();
a.b = b;
b.a = a;
a = null;
b = null;
```

logički ne postoje
tehnički postoje
curenje memorije

liste, grafovi,
observer pattern

ilustracija; moderni JVM neće imati problem - u nastavku

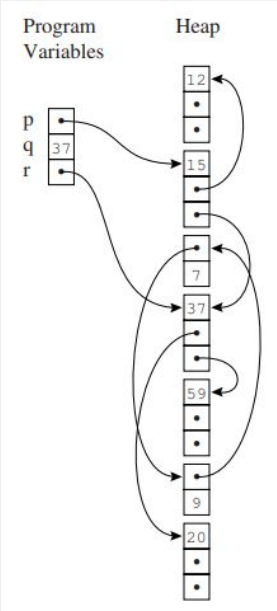
trace-based GC

Objekat je živ ako je dostižan.

- Krene se od početnog skupa objekta (GC roots) i prate se sve reference.
- Kad se formira skup dostupnih objekata – svi ostali su kandidati za brisanje
- mark-and-sweep

- u jednom momentu se saznaje status za sve objekte; prilika za manju fragmentaciju
- efikasnost: svi moraju da se markiraju → određene pauze

- ciklus može biti nedostižan bez obzira što je ciklus, što u njemu postoje reference



Appel, Andrew W. "Modern compiler Implementation in C."

Uopšteno:

- Različiti (hibridni) pristupi, algoritmi, poboljšanja

- kad se markiraju, objekti se pomeraju tako da svi dostupni objekti budu spojeni. Izbegava se fragmentacija.
  Memorija, u startu, može biti podeljena na dva dela; samo sa jednim delom se radi;

- Incremental GC - ne mora odjednom sve da se odradi

- Generational GC - ko preživi dovoljno dugo neće biti dealociran

- mehanizmi u PL: soft, strong i weak reference; finalize metode...

- više niti, više procesora?

Generational hypothesis

Većina objekata umire veoma brzo, dok mali broj objekata živi veoma dugo.

- nema smisla svaki put skenirati ceo heap
- treba se fokusirati na deo memorije gde se “najviše dešava”
- heap se deli na generacije
  - young generation sadrži novokreirane objekte
  - old generation sadrži objekte koji su preživeli više GC ciklusa

JVM Heap

Young Gen (Nursery) - novo-alocirani objekti

Eden → posle Minor GC: Survivor 0 ili Survivor 1

posle više GC: Old Gen

posle Major GC → prazni se Old Gen

JIT i GC

U JVM-u GC ne radi izolovano. On sarađuje sa JIT kompajlerom koji ima dubok uvid u ponašanje programa.

Escape Analysis

Objekat beži (escape-uje) ako postane vidljiv van metode (ili niti) u kojoj je kreiran
Objekat ne beži ako:

- ne izlazi iz metode
- ne vraća se kao rezultat
- ne čuva se u polju ili statičkoj promenljivoj

→

izbeći alokaciju na heap-u;

- alocirati objekat na steku ili
- scalar replacement (raspakovan na stek i/ili registre)

→ JIT smanjuje pritisak na GC, a programer piše kao da se svi alociraju na hipu i podležu GC